Advanced Applications of Statistical Packages
Report 1

Author 1, Author 2

May 17, 2026

Contents

Table 1: Summary of Iris data set

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
	Min. :4.300	Min. :2.000	Min. :1.000	Min. :0.100	Length:150
	1st Qu.:5.100	1st Qu.:2.800	1st Qu.:1.600	1st Qu.:0.300	Class :character
	Median :5.800	Median :3.000	Median :4.350	Median :1.300	Mode :character
	Mean :5.843	Mean :3.054	Mean :3.759	Mean :1.199	NA
	3rd Qu.:6.400	3rd Qu.:3.300	3rd Qu.:5.100	3rd Qu.:1.800	NA
	Max. :7.900	Max. :4.400	Max. :6.900	Max. :2.500	NA

1 Chunks

We write R codes in chunks. Between << and >>= we can put its name and local options. The global options can be given in the preamble. For example echo=TRUE shows (prints in the output PDF file) the results of the code, while echo=FALSE hides them. Another examples of chunk options can be found in the code of this file below. You need to remember that each chunk must have different name. If the name is duplicated, the file may not compile, returning following error:

```
processing file:  FileName.rnw
Error in parse block(g[-1], g[1], params.src):  duplicate label 'ChunkName'
Calls:  knit process file split file lapply FUN parse block
Execution halted
```

2 Simple data analysis

We can import data set from a file or from URL using `read.csv` function.

```
# 1. Importing data set from URL (CSV format)
url <- "https://archive.ics.uci.edu/ml/machine-learning-databases/iris/iris.data"
# Manually adding columns names
iris_data <- read.csv(url, header = FALSE,
                      col.names = c("Sepal.Length", "Sepal.Width",
                                    "Petal.Length", "Petal.Width", "Species"))
```

A following code can be used to produce a simple table.

```
summary_table <- summary(iris_data)
kable(summary_table,
      format = "latex",
      caption = "Summary of Iris data set")
```

Each table should have a caption above and it should be mentioned in the text.

```r
plot(iris_data$Petal.Length, iris_data$Petal.Width,
     col = as.factor(iris_data$Species), # Colouring by species
     pch = 19,                           # point type - filled circle
     main = "Plot of dependencies of petal length and width",
     xlab = "Petal length",
     ylab = "Petal width")

# list of points types:
# https://www.sthda.com/english/wiki/r-plot-pch-symbols-the-different-point-shapes-ava

pl_mean = round(mean(iris_data$Petal.Length), 3)

# Adding legend
legend("topleft", legend = unique(iris_data$Species),
       col = 1:length(unique(iris_data$Species)), pch = 19)
```

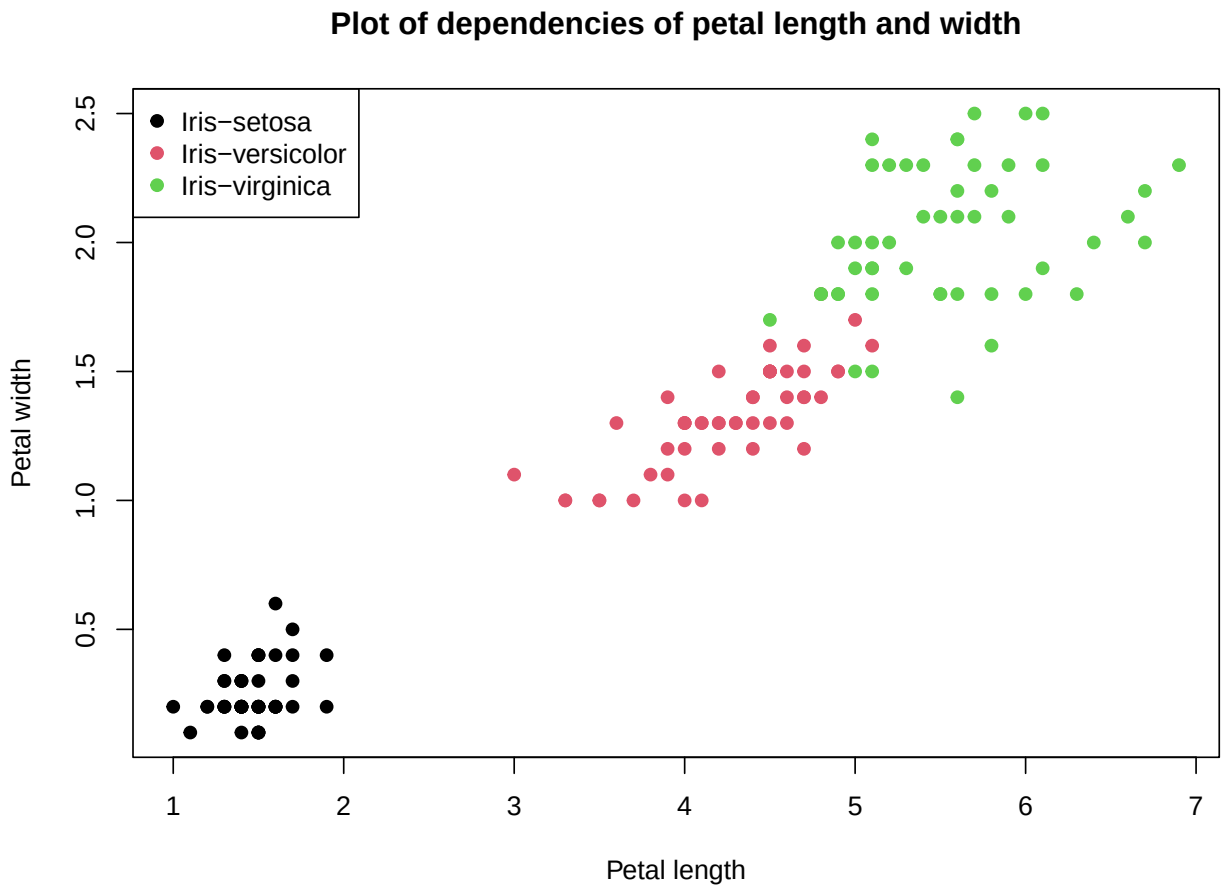

Figure 1: Plot of dependencies of petal length and width

Each figure should have a caption below. In the text there should be a reference to each figure. Simple plots for quick visualisations can be produced using native R functions `plot`, `curve`, `lines` or `boxplot`, but in a report it is required to use professional packages such as `ggplot2`. A function `ggplot` requires that its input is given as an object of type *data.frame* (or

tibble). For a ggplot2 cheat sheet, see, for example, https://miro.medium.com/v2/resize:fit:4800/format:webp/1*JXPvSBL-DxocIjdziCJkfw.jpeg.

We can display the values of the variables computed in R chunks. For example, the average length of the petals in Iris data is 3.759. We can also performs simple computations inside *Sexpr*, for example: 3.759.

R language has many functions which will be useful in this course, including:

- Basic statistics: *mean*, *median*, *sum*, *max* i *min*,
- The *which* function returns a vector containing indices where the argument is `TRUE`. It can be useful, for example, for finding the index of a vector's maximum, although you can also directly use the *which.max* or which.min functions.
- Finding the solution to an equation: *uniroot* (Usage examples: [URL]).
- Finding extrema of multivariable functions: *optim* (Usage examples: [URL]).
- Creating arithmetic sequences: *seq*.
- Creating lists (*list*), vectors (*c*), matrices (*matrix*), and data frames (*data.frame*).
- Functions for changing data types: *as.data.frame*, *as.numeric*, *as.character*, *toString*.
- Creating a table (or matrix or data frame) by binding rows (*rbind*) or columns (*cbind*) and assigning labels to rows (*rownames*) and columns (*colnames*).
- Functions returning object sizes: *length* (vector length), *nrow* (number of rows), *ncol* (number of columns), and *dim* (number of rows and columns).

3 Tips for creating reports

Reports can be prepared using Sweave (rnw file) or Markdown (Rmd file). The report should be prepared in PDF format. The advantage of Sweave is its integration with LaTeX, while the advantage of Markdown is its flexibility, thanks to which it can also be used to produce slide shows or HTML files. When creating your first report, you can use the attached template. It is also worth following the tips provided below.

3.1 LaTeX Notes

- When writing about variables in the text, e.g., sample X or sample size n, use mathematical mode, i.e., write X and not X.
- Functions (e.g., exp or min) and operators (e.g., lim) should be written using a backslash, i.e., do not write exp or lim, but \exp or \lim.
- It is better to write parentheses in the form `\left( ... \right)` rather than (...), because then their height adjusts to the size of the content.
- English inserts (in case of reports written in different languages), function names, and package names should be written in italics.

3.2 Writing code

- The report should include the code of functions in which calculations essential to solving the problem are performed. It is not necessary to include code used for generating plots, tables, etc. You also do not need to include auxiliary code used, for example, for transforming data frames with results.
- Reports should not contain information about errors and warnings. You can disable the insertion of such messages by placing `error=FALSE, warning=FALSE` in the chunk options or global options. Warnings usually concern library versions and can be ignored, whereas errors indicate that some part of our code failed to compile.
- Similarly, if a function "prints" unnecessary messages, they can be disabled by placing `message=FALSE` in the chunk options or global options. For some functions, you can also disable printing messages by calling the function with the argument `print=FALSE`.
- Unless specified that ready-made libraries can be used, solve the problems by writing functions yourself from scratch.
- Code placed in gray boxes (chunks) should not exceed the document margin. If it goes beyond the margin, you need to break the line. The limit is 79 characters (this may vary depending on font size or other factors).
- The code used for calculations does not have to be written in an optimal way, but it should not be written in an overly complex manner either.
- To minimize the risk of error, it is recommended that when performing complex calculations, you do not create dozens of separate variables for each set of considered parameters. It is recommended to keep each result in a single vector (or matrix). By creating a separate variable for each value placed in a table, you run the risk of making a mistake when entering variable names into the results matrix.

3.3 Optimality of the code

The fastest way for computing is using functions like *colMeans*, *colSums*, *rowMeans* and *rowSums* if possible. A bit slower are the apply-type functions, such as *sapply*, *lapply* etc. Using *for* loops is the slowest option. Paralleling may increase the speed of the computations. This is mostly important in case of simulation experiments or analysis of large data.

3.4 Plots

- Plots can be created in R using the *ggplot2* package or using standard built-in functions in R (e.g., *plot*, *points*, *curve*, and *lines*). In the reports it is required to use *ggplot2* or other professional package.
- Figure captions are placed below the figures. A caption should not end with a period.
- The caption should contain full information about the content of the plot; therefore, we should not write "CDF" or "Plot of CDF," but rather, e.g., "Plot of the cumulative distribution function of the exponential distribution with parameter $\lambda = 4$".

- Every plot should contain a label (e.g., \label{fig:ExpCDF}). Every figure should be referenced in the text (e.g., \ref{fig:ExpCDF}).
- Both the horizontal and vertical axes should be labeled. Pay attention to whether the values on the vertical axis are correct, i.e. there should have range beyond minimum and maximum of the values presented on axis Y.
- The plot should be large enough to read the information easily, but small enough not to take up the entire page. Sometimes it is more useful to create an illustration consisting of several small plots (an example can be found in the report template).

3.5 Tables

- Numbers in the table should be formatted in such a way that the results are legible and essential information can be read from it. For example, if the difference in the compared numbers is visible only at the fourth decimal place, we should display the results with 4 or 5 decimal places, rather than rounding to two decimal places.
- If natural numbers are placed in a table (e.g., sample size or the number of observations having a certain characteristic), the column with such numbers should be formatted so that it does not display decimal places; thus, we do not write the number 10 as 10.0000.
- Table captions are placed above the tables. A caption should not end with a period.
- As with plots, the caption should contain full information about the content of the table. The table should contain a label and should be referenced in the text.
- Care should be taken not to incorrectly reference formulas, tables, and figures in the text ? this results in **??** appearing in the text. This may happen if we change the *label* or if we place it in the wrong location.

3.6 Inference

- If you see that you have obtained nonsensical results, do not include them in the report; instead, keep trying until you obtain sensible results. Usually, we know what the result should be based on intuition or classroom discussions.
- You should not include conclusions in the report that contradict the results presented in the table or on the plot.
- Solutions to problems involving simulations or data analysis should include conclusions that can be drawn from that analysis.
- In the case of hypothesis testing, it is necessary to clearly state what the null and alternative hypotheses are. Furthermore, it is not enough to write the p-value; you must also write what decision should be made based on the obtained result.